生产车间照片——研发环境与过程说明

> **重要说明**：本项目为纯软件系统研发，不存在传统制造业意义上的"生产车间"、"工艺设备"、"生产线"等物理形态。根据专利局"如没有，就写一个过程"的指导意见，以下详细描述本项目的研发环境、开发流程和迭代过程，以替代生产车间相关材料。

一、研发环境说明

1.1 研发场所

本项目为软件研发项目，研发活动主要在以下场所开展：

- **个人开发工作站**：Windows 11 操作系统，配备 NVIDIA GPU，用于代码编写、本地测试与模型推理调试
- **实验室环境**：可部署 Linux 服务器的实验室环境，用于系统集成测试和模型运行

1.2 "生产线"对应的软件研发流程

软件研发的"生产线"对应的是以下开发与部署流水线：

```
需求分析 → 系统设计 → 编码实现 → 单元测试 → 集成测试 → 部署运行 → 用户反馈 → 迭代优化
    |          |          |          |          |          |          |
    ▼          ▼          ▼          ▼          ▼          ▼          ▼
 需求文档    架构图纸    源代码    测试用例    测试报告    部署配置    用户报告    新版本发布
```

1.3 研发"车间"对应关系

制造业概念	软件研发对应	本项目工具/平台
生产车间	开发环境	VS Code + Python venv + Node.js
生产线	CI/CD 流水线	Git + 代码审查 + 手动部署
工艺设备	开发工具链	FastAPI + React + MySQL + PyTorch
原材料	开源组件与数据	MUSDB18 数据集、预训练模型权重
半成品	模块/组件	各 API 路由、React 组件
成品	可运行系统	AudioLab v1.0
质检	测试	手动功能测试 + API 测试

二、研发过程详细时间线

2.1 第一阶段：创意产生与需求分析（2025年底）

场景描述：在自研音乐源分离算法的过程中，面临实验管理混乱的现实困难。每次进行对比实验都需要手动组织文件、用 Excel 记录数据、用 Python 脚本单独画图，整个流程耗时且易出错。在一次团队讨论中，提出了"能否做一个统一管理实验的平台"的想法。

关键活动：

- 梳理实验管理中的痛点
- 调研现有工具和平台的不足
- 在笔记本/白板上画出初步的系统构想
- 确定核心技术栈选型

产出物：

- 手绘的系统构想草图
- 技术选型对比表
- 核心需求清单

2.2 第二阶段：核心架构搭建（2025年底）

场景描述：确定方案后，开始搭建项目骨架。最初只关注"能把分离任务跑起来"这个最核心的功能。先搭建了 FastAPI 后端框架，定义了基本的数据库模型，然后逐步加入任务管理和文件处理逻辑。

关键活动：

- 搭建 FastAPI 项目结构
- 设计数据库 ER 模型（用户、任务、评分等实体及关系）
- 实现雪花 ID 生成器
- 实现 JWT 认证与 RBAC 权限控制
- 搭建 React 前端框架
- 实现音频上传与分离任务创建的基本流程

产出物：

- 可运行的后端 API 服务（初始版本）
- 基础前端页面
- 数据库建表脚本

2.3 第三阶段：分离引擎与指标（2026年年初）

场景描述：这是项目技术难度最高的阶段。需要解决两个核心问题：如何统一调用不同的分离算法，以及如何正确实现 BSS Eval 多源评估框架的指标计算。

在最开始设计分离引擎时，尝试过在 Python 进程内直接 import 模型进行推理。但很快发现不同模型对 PyTorch 版本、依赖包的版本要求不同，进程内推理会导致依赖冲突。经过几次尝试后，改为子进程调用的方案，每个分离任务通过独立的子进程运行 `inference.py`，进程间通过文件系统交换数据。这个设计决策避免了依赖冲突问题，也使得引擎可以方便地扩展（增加新模型只需增加环境变量配置）。

在实现客观指标时，仔细研读了 Vincent et al. (2006) 的 BSS Eval 论文，按照论文中的投影矩阵定义，逐步实现了多源分解算法。在调试阶段，用已知的混合信号（手动加权合成的简单信号）验证了计算结果的正确性。

关键活动：

- 实现 MSSLocalEngine（子进程调用 inference.py）
- 实现 MockSeparationEngine（ffmpeg 滤波器模拟）
- 实现任务队列管理器（单例、异步 FIFO）
- 实现任务取消机制（全局取消标记 + 进程终止）
- 实现 BSS Eval v3 多源指标计算
- 实现 SI-SDR 计算
- 提出并实现 WOC/SOC 重叠一致性指标
- 调试和验证指标计算正确性

产出物：

- 可用的分离引擎（3 种模式）
- 完整的客观指标计算模块
- WOC/SOC 指标的 Python 实现

2.4 第四阶段：主观评价与统计分析（2026年初）

场景描述：客观指标虽然能量化分离算法的性能，但在实际听感上，"指标好不代表听着好"。为了建立这个连接，设计了基于 MUSHRA 规范的主观评价流程，并在平台中实现了完整的评分系统。

在实现统计分析模块时，考虑到引入 SciPy 这样的重依赖会增加部署复杂度，决定从零实现所需的统计方法。通过查阅统计学教材和论文，用纯 Python + NumPy 实现了 Spearman 等级相关、Wilcoxon 符号秩检验和排列检验。在打结（tied ranks）处理上，采用了平均秩的方法；在 Wilcoxon 检验中，使用正态近似计算 p 值，并在存在打结时进行方差修正。

关键活动：

- 设计并实现主观评分系统（三维度、批量提交）
- 实现 Spearman 等级相关系数计算
- 实现排列检验（Permutation Test）
- 实现 Wilcoxon 符号秩检验（含效应量）
- 实现主客观联合数据查询
- 实现 SOC-音质相关性分析链路
- 前端图表展示（Recharts 雷达图、散点图、箱线图）

产出物：

- 完整的主观评分模块
- 统计分析模块（3 种统计方法）
- 主客观联合分析 API
- 前端数据可视化组件

2.5 第五阶段：系统完善与数据收集（2026年中）

场景描述：核心功能稳定后，开始补充系统管理功能（日志审计、系统配置、通知中心、数据仪表盘），完善音频资源库，并基于 MUSDB18 数据集运行完整的对比实验。同时整理了研发过程中的设计文档、架构图和实验方案，撰写了学术论文。

关键活动：

- 系统管理与审计功能开发
- 音频资源库完善
- 通知中心与消息系统开发
- 数据统计仪表盘开发
- 基于 MUSDB18 运行完整对比实验
- 邀请专家进行主观评分
- 整理实验数据与撰写论文
- 系统界面优化

产出物：

- 功能完整的 AudioLab 系统
- 对比实验原始数据
- 学术论文初稿
- 系统使用说明书
- 本专利申报材料

三、研发过程中的关键决策记录

时间节点	决策事项	背景与理由
2025年底	选用 FastAPI 而非 Flask/Django	需要异步支持（任务队列），FastAPI 原生支持 asyncio
2025年底	采用前后端分离架构	前端可独立演进，后端 API 可复用于不同客户端
2025年底	分离引擎采用子进程调用	避免 PyTorch 依赖冲突，隔离不同模型的运行环境
2025年底	任务队列采用单任务串行	GPU 显存有限，并行运行易 OOM
2026年初	统计方法从零实现	避免引入 SciPy 重依赖，降低部署复杂度
2026年初	提出 WOC/SOC 辅助指标	发现可视化与客观指标的差异，需要桥接解释
2026年中	主观评价参考 MUSHRA 规范	该规范是 ITU 推荐的主观音质评价标准，具有权威性

时间节点	决策事项	背景与理由
2026年中	使用 Recharts 做前端图表	轻量、React 原生、支持多种图表类型

四、"生产"过程中的检验记录

软件开发的"质量检验"主要通过以下方式完成：

检验方式	说明
功能测试	逐个 API 端点手动测试，确保请求/响应正确
流程测试	端到端走通完整的"注册→上传→分离→评分→分析"流程
指标验证	用已知混合比例的人工信号验证 BSS Eval 指标计算结果
UI 走查	逐页检查前端界面的正确性和交互体验
错误处理	测试各种异常场景（文件不存在、模型加载失败、网络超时等）的提示与恢复